33.1 CLI 架构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~38k tokens，输出 ~5k tokens（本节）

OpenClaw 的命令行界面（CLI）是用户与系统交互的主要入口。从 `openclaw gateway` 启动服务，到 `openclaw agent` 发送消息，再到 `openclaw doctor` 诊断问题——所有操作都经由 CLI 分发。本节从三个维度拆解 CLI 的架构：Commander.js 命令框架、命令分发机制、命令格式化工具。

33.1.1 Commander.js 命令框架（`src/cli/program/build-program.ts`）

Commander.js 简介

> **衍生解释 — Commander.js**
>
> Commander.js 是 Node.js 生态中最流行的命令行框架，被 npm、webpack 等知名工具采用。它提供了一套声明式 API 来定义命令、选项、参数和帮助文本。核心概念包括：
>
> - **Command** — 一个命令节点，可以包含子命令（形成树状结构）
> - **Option** — 以 `--name` 或 `-n` 形式传入的命名参数
> - **Argument** — 位置参数（positional argument）
> - **Action** — 命令被匹配时执行的回调函数
> - **Hook** — 在命令执行前后触发的钩子（如 `preAction`、`postAction`）

程序构建入口

`buildProgram()` 是整个 CLI 的工厂函数，负责把一棵完整的命令树组装起来：

```typescript
// src/cli/program/build-program.ts
import { Command } from "commander";
import { registerProgramCommands } from "./command-registry.js";
import { createProgramContext } from "./context.js";
import { configureProgramHelp } from "./help.js";
import { registerPreActionHooks } from "./preaction.js";

export function buildProgram() {
  const program = new Command();
  const ctx = createProgramContext();
  const argv = process.argv;

  configureProgramHelp(program, ctx);       // ① 帮助文本与全局选项
  registerPreActionHooks(program, ctx.programVersion); // ② 前置钩子
  registerProgramCommands(program, ctx, argv);         // ③ 注册所有命令

  return program;
}
```

四步构建流程：

步骤	函数	职责
① 创建上下文	`createProgramContext()`	收集版本号、支持的通道列表（WhatsApp/Telegram/Discord/...）
② 配置帮助	`configureProgramHelp()`	设置 `--dev` 、 `--profile` 、 `--no-color` 全局选项；自定义帮助输出格式（语法高亮、示例代码）
③ 注册前置钩子	`registerPreActionHooks()`	在每个命令执行前：输出 Banner、设置 verbose、确保配置有效、加载插件
④ 注册命令	`registerProgramCommands()`	遍历命令注册表，注册所有顶层命令和子命令

ProgramContext — 上下文对象

```
// src/cli/program/context.ts
export type ProgramContext = {
  programVersion: string;      // 当前版本号（如 "2.4.0"）
  channelOptions: string[];    // 支持的通道名称列表
  messageChannelOptions: string; // "whatsapp|telegram|discord|..."
  agentChannelOptions: string;   // "last|whatsapp|telegram|..."
};
```

`agentChannelOptions` 在通道列表前额外加了 `"last"`，让用户可以用 `--channel last` 自动选上次用过的通道。

帮助系统定制

OpenClaw 对 Commander.js 的默认帮助系统做了不少定制：

```
// src/cli/program/help.ts（简化）
program.configureHelp({
  sortSubcommands: true,   // 子命令按字母排序
  sortOptions: true,       // 选项按字母排序
  optionTerm: (option) => theme.option(option.flags),   // 选项名着色
  subcommandTerm: (cmd) => theme.command(cmd.name()),    // 命令名着色
});

program.configureOutput({
  writeOut: (str) => {
    // 为 "Usage:"、"Options:"、"Commands:" 标题着色
    const colored = str
      .replace(/^Usage:/gm, theme.heading("Usage:"))
      .replace(/^Options:/gm, theme.heading("Options:"))
      .replace(/^Commands:/gm, theme.heading("Commands:"));
    process.stdout.write(colored);
  },
});
```

帮助文本还附带了丰富的示例（Examples），每条示例由命令和描述两行组成，命令中的 `openclaw` 会被 `replaceCliName()` 替换为实际的 CLI 名称。

preAction 前置钩子

Commander.js 的 `preAction` 钩子在**每个命令的 action 执行前**解发：

```
// src/cli/program/preaction.ts（简化）
program.hook("preAction", async (_thisCommand, actionCommand) => {
  // 1. 设置进程标题（如 "openclaw-gateway"）
  setProcessTitleForCommand(actionCommand);

  // 2. 跳过帮助/版本命令
  if (hasHelpOrVersion(argv)) return;

  // 3. 输出 Banner（除 update、completion 等命令外）
  if (!hideBanner) emitCliBanner(programVersion);

  // 4. 设置 verbose 模式
  setVerbose(getVerboseFlag(argv, { includeDebug: true }));

  // 5. 确保配置文件有效
  await ensureConfigReady({ runtime, commandPath });

  // 6. 为需要通道的命令加载插件
  if (PLUGIN_REQUIRED_COMMANDS.has(commandPath[0])) {
    ensurePluginRegistryLoaded();
  }
});
```

`ensureConfigReady` 是个重要的守卫函数——配置文件存在但无效（JSON 解析失败或 schema 校验不通过）时，它会打印错误并提示用户运行 `openclaw doctor --fix` 。不过 `doctor` 、 `health` 、 `status` 等诊断命令被列入白名单，即使配置无效也能跑。

命令注册表

所有顶层命令通过 `commandRegistry` 数组集中管理：

```
// src/cli/program/command-registry.ts（简化）
export const commandRegistry: CommandRegistration[] = [
  { id: "setup",      register: ({ program }) => registerSetupCommand(program) },
  { id: "onboard",    register: ({ program }) => registerOnboardCommand(program) },
  { id: "configure",  register: ({ program }) => registerConfigureCommand(program) },
  { id: "config",     register: ({ program }) => registerConfigCli(program) },
  { id: "maintenance", register: ({ program }) => registerMaintenanceCommands(program)
  { id: "message",    register: ({ program, ctx }) => registerMessageCommands(program, c
  { id: "memory",     register: ({ program }) => registerMemoryCli(program), routes: [..
  { id: "agent",      register: ({ program, ctx }) => registerAgentCommands(program, ctx
  { id: "subclis",    register: ({ program, argv }) => registerSubCliCommands(program, a
  { id: "status-health-sessions", register: ({ program }) => registerStatusHealthSessio
  { id: "browser",    register: ({ program }) => registerBrowserCli(program) },
];
```

每个注册项包含：

- **id** — 唯一标识符，用于调试和日志
- **register** — 接收 `{ program, ctx, argv }` 的注册函数
- **routes**（可选）— 快速路由规则（详见下一小节）

注意 `subclis` 这一项——它负责注册 25 个延迟加载的子命令（gateway、models、channels 等），是 CLI 命令量最大的来源。

33.1.2 命令分发（`src/cli/run-main.ts`）

两阶段分发架构

OpenClaw CLI 的命令分发采用了一个**两阶段架构**：

```
用户输入 argv
    │
    ├── 阶段 1：快速路由（tryRouteCli）
    │    └── 匹配 → 直接执行，跳过 Commander.js
    │
    └── 阶段 2：完整分发（Commander.js parseAsync）
         ├── 懒加载子命令（registerSubCliByName）
         ├── 注册插件命令（registerPluginCliCommands）
         └── program.parseAsync(argv)
```

为什么要两阶段？性能。完整构建 Commander.js 程序需要导入大量模块、加载配置、注册几十个命令。而像 `openclaw status` 这样的高频命令，只需要很少的依赖就能跑。快速路由让这些命令绕过整个 Commander.js 管道。

完整启动流程

```
// src/cli/run-main.ts（简化）
export async function runCli(argv: string[] = process.argv) {
  // ① Windows 兼容：清理 argv 中多余的 node.exe 路径
  const normalizedArgv = stripWindowsNodeExec(argv);

  // ② 加载 .env 文件
  loadDotEnv({ quiet: true });

  // ③ 规范化环境变量
  normalizeEnv();

  // ④ 确保 openclaw 可执行文件在 PATH 中
  ensureOpenClawCliOnPath();

  // ⑤ 检查 Node.js/Bun 版本要求
  assertSupportedRuntime();

  // ⑥ 快速路由：尝试直接处理已知命令
  if (await tryRouteCli(normalizedArgv)) return;

  // ⑦ 开启结构化日志捕获
  enableConsoleCapture();

  // ⑧ 懒加载 Commander.js 程序
  const { buildProgram } = await import("./program.js");
  const program = buildProgram();

  // ⑨ 安装全局错误处理
  installUnhandledRejectionHandler();

  // ⑩ 重写 --update → update（向后兼容）
  const parseArgv = rewriteUpdateFlagArgv(normalizedArgv);

  // ⑪ 按需注册子命令（懒加载）
  const primary = getPrimaryCommand(parseArgv);
  if (primary) {
    const { registerSubCliByName } = await import("./program/register.subclis.js");
    await registerSubCliByName(program, primary);
  }

  // ⑫ 注册插件贡献的命令
```

```
  if (!shouldSkipPluginRegistration) {
    const { registerPluginCliCommands } = await import("../plugins/cli.js");
    registerPluginCliCommands(program, loadConfig());
  }

  // ⑬ 执行命令
  await program.parseAsync(parseArgv);
}
```

快速路由（Fast-Path Routing）

快速路由是个性能优化机制，让特定命令绕过完整的 Commander.js 构建流程：

```
// src/cli/route.ts
export async function tryRouteCli(argv: string[]): Promise<boolean> {
  // 环境变量可以禁用快速路由
  if (isTruthyEnvValue(process.env.OPENCLAW_DISABLE_ROUTE_FIRST)) return false;
  // 帮助/版本请求需要完整的命令树
  if (hasHelpOrVersion(argv)) return false;

  const path = getCommandPath(argv, 2);  // 提取最多 2 级命令路径
  if (!path[0]) return false;

  const route = findRoutedCommand(path);  // 从注册表查找匹配的路由
  if (!route) return false;

  // 输出 Banner + 确保配置就绪
  await prepareRoutedCommand({ argv, commandPath: path, loadPlugins: route.loadPlugins

  return route.run(argv);  // 直接执行
}
```

支持快速路由的命令包括：

命令	路由路径	说明
`openclaw health`	`["health"]`	健康检查
`openclaw status`	`["status"]`	状态查询
`openclaw sessions`	`["sessions"]`	会话列表
`openclaw agents list`	`["agents", "list"]`	Agent 列表

命令	路由路径	说明
`openclaw memory status`	`["memory", "status"]`	记忆状态

每个路由的 `run()` 函数直接解析 argv 中的标志位（`--json`、`--verbose`、`--timeout` 等），无需 Commander.js 参与。如果 argv 格式不符合预期（例如 `--timeout` 后没有值），路由返回 `false`，交由完整流程处理。

懒加载子命令

CLI 注册了 25 个子命令（gateway、models、channels、nodes、skills、cron、sandbox、tui 等），但每次执行只会用其中一个。全部加载会拖慢启动——每个子命令文件可能还会引入各自的依赖链。

OpenClaw 的解决方案是**三层懒加载**：

```
层级 1: 仅注册目标子命令（最常见路径）
  ↓ 找不到
层级 2: 注册占位符命令（延迟加载）
  ↓ 占位符被触发
层级 3: 替换占位符，加载真实命令，重新解析
```

核心实现在 `register.subclis.ts`：

```
// 层级 1：如果已知目标命令，直接注册它
const primary = getPrimaryCommand(argv);
if (primary && shouldRegisterPrimaryOnly(argv)) {
  const entry = entries.find((e) => e.name === primary);
  if (entry) {
    registerLazyCommand(program, entry);
    return;  // 只注册一个命令
  }
}

// 层级 2：注册所有命令的占位符
for (const entry of entries) {
  registerLazyCommand(program, entry);
}
```

占位符命令的精妙之处在于——它是个真实的 Commander.js 命令，但设置了 `allowUnknownOption(true)` 和 `allowExcessArguments(true)`，无论用户传什么参数都能匹配。当占位符的 `action` 被触发时：

```
function registerLazyCommand(program: Command, entry: SubCliEntry) {
  const placeholder = program.command(entry.name).description(entry.description);
  placeholder.allowUnknownOption(true);
  placeholder.allowExcessArguments(true);
  placeholder.action(async (...actionArgs) => {
    // 1. 移除占位符
    removeCommand(program, placeholder);
    // 2. 加载并注册真实命令
    await entry.register(program);
    // 3. 重新构建 argv 并再次解析
    const parseArgv = buildParseArgv({ programName, rawArgs, fallbackArgv });
    await program.parseAsync(parseArgv);
  });
}
```

这种设计让 `openclaw gateway start` 只需加载 `gateway-cli.js` ，不会碰 `models-cli.js` 、 `channels-cli.js` 等无关模块。

> **衍生解释 — 懒加载（Lazy Loading）**
>
> 懒加载是一种延迟初始化模式：只在真正需要某个资源时才加载它。在 CLI 场景中，这意味着只导入（import）用户实际请求的命令模块。JavaScript 的动态 `import()` 语法使这一模式变得简洁： `const mod = await import("./gateway-cli.js")` 。相比在文件顶部的静态 `import` ，动态导入只在执行到该语句时才触发模块加载。

argv 解析工具

`argv.ts` 提供了一套独立于 Commander.js 的 argv 解析工具，供快速路由和早期决策使用：

```
// 提取命令路径（跳过标志位）
getCommandPath(["node", "openclaw", "memory", "--verbose", "status"], 2)
// → ["memory", "status"]

// 提取主命令
getPrimaryCommand(["node", "openclaw", "gateway", "start"])
// → "gateway"

// 检查帮助/版本标志
hasHelpOrVersion(["node", "openclaw", "--help"])
// → true

// 获取标志值（支持 --name value 和 --name=value 两种语法）
getFlagValue(["node", "openclaw", "--timeout", "5000"], "--timeout")
// → "5000"
```

`getCommandPath` 的实现跳过所有以 `-` 开头的参数（标志位），只收集位置参数，遇到 `--` （标志终止符）时停止。这是 UNIX 命令行的标准惯例。

Windows 兼容性

`stripWindowsNodeExec` 处理 Windows 平台的一个边缘问题——某些 Windows 环境下， `process.argv` 可能包含多余的 `node.exe` 路径或含控制字符（ASCII 码 < 32）的参数：

```
// 原始 argv（Windows 异常情况）：
["C:\\nodejs\\node.exe", "C:\\nodejs\\node.exe", "C:\\openclaw\\openclaw", "gateway"]

// 清理后:
["C:\\nodejs\\node.exe", "C:\\openclaw\\openclaw", "gateway"]
```

函数还会清理引号包裹和 Windows 长路径前缀（ `\\?\` ）。

33.1.3 命令格式化（ `src/cli/command-format.ts` ）

当 OpenClaw 需要在日志、错误消息或帮助文本中展示 CLI 命令时， `formatCliCommand` 确保命令使用正确的名称和配置：

```
// src/cli/command-format.ts
export function formatCliCommand(command: string): string {
  const cliName = resolveCliName();      // 解析实际 CLI 名称
  let formatted = replaceCliName(command, cliName); // 替换 "openclaw" 为实际名称

  // 如果设置了 OPENCLAW_PROFILE 环境变量，自动注入 --profile 标志
  const profile = process.env.OPENCLAW_PROFILE;
  if (profile) {
    formatted = formatted.replace(cliName, `${cliName} --profile ${profile}`);
  }

  return formatted;
}
```

CLI 名称解析

`resolveCliName` 从 `process.argv[1]` （脚本路径）的文件名中提取 CLI 名称。如果是已知名称（`"openclaw"`）就直接用，否则回退到默认值：

```
// src/cli/cli-name.ts
export function resolveCliName(argv: string[] = process.argv): string {
  const base = path.basename(argv[1] ?? "").trim();
  if (KNOWN_CLI_NAMES.has(base)) return base;
  return DEFAULT_CLI_NAME; // "openclaw"
}
```

`replaceCliName` 用正则表达式 `/^(?:(pnpm|npm|bunx|npx)\s+)?(openclaw)\b/` 匹配命令前缀，保留包管理器前缀（如 `npx openclaw` → `npx mycli`），只替换 CLI 名称部分。

Profile 注入

OpenClaw 支持多 Profile 运行——通过 `--profile <name>` 或 `OPENCLAW_PROFILE` 环境变量指定。内置的 `--dev` 选项实际上是个名为 `dev` 的预设 Profile，会把状态目录隔离到 `~/.openclaw-dev`，Gateway 端口偏移到 19001。

当 `OPENCLAW_PROFILE` 环境变量被设置时，`formatCliCommand` 自动在所有格式化的命令中注入 `--profile` 标志，确保用户看到的命令示例可以直接复制执行。

CLI 依赖注入

`deps.ts` 定义了 CLI 的通道发送依赖接口：

```
// src/cli/deps.ts
export type CliDeps = {
  sendMessageWhatsApp: typeof sendMessageWhatsApp;
  sendMessageTelegram: typeof sendMessageTelegram;
  sendMessageDiscord:  typeof sendMessageDiscord;
  sendMessageSlack:    typeof sendMessageSlack;
  sendMessageSignal:   typeof sendMessageSignal;
  sendMessageIMessage:  typeof sendMessageIMessage;
};
```

`createDefaultDeps()` 返回真实实现，`createOutboundSendDeps()` 将其转换为 `OutboundSendDeps` 接口，解耦了 CLI 层与通道发送层的命名约定。这种依赖注入模式让测试时可以替换为 mock 实现。

Banner 输出

CLI 启动时会输出一条品牌标识行，包含版本号、Git commit hash 和随机标语（tagline）：

```
🦞 OpenClaw 2.4.0 (a1b2c3d) — Your AI, your rules
```

Banner 的输出受多重条件控制：

- **非 TTY** → 不输出（管道场景）
- **`--json` 标志** → 不输出（机器可读输出）
- **`--version` 标志** → 不输出
- **`OPENCLAW_HIDE_BANNER` 环境变量** → 不输出
- **`update` / `completion` 命令** → 不输出
- **已输出过** → 不重复输出（`bannerEmitted` 单例标志）

在富终端（Rich TTY）模式下，Banner 会用 ANSI 颜色代码着色；普通终端下输出纯文本。如果终端宽度不够，会自动折为两行。

本节小结

1. **Commander.js 框架**：OpenClaw 基于 Commander.js 构建命令树，通过 `buildProgram()` 工厂函数组装——创建上下文、配置帮助、注册前置钩子、注册命令四步完成
2. **两阶段分发**：快速路由（`tryRouteCli`）允许高频命令绕过完整的 Commander.js 管道直接执行；只有快速路由未匹配的命令才进入完整流程
3. **三层懒加载**：25 个子命令按需加载——优先注册目标命令，其余使用占位符延迟加载，占位符被触发时替换为真实命令并重新解析
4. **12 步启动流程**：从 Windows 兼容处理到 `.env` 加载、运行时检查、快速路由、懒导入、子命令注册、插件命令注册，最终执行 `parseAsync`
5. **命令格式化**：`formatCliCommand` 确保日志和帮助中的命令示例使用正确的 CLI 名称并注入 Profile 标志，保证用户可以直接复制执行
6. **依赖注入与 Banner**：CLI 层通过 `CliDeps` 类型解耦通道发送实现；Banner 输出在多种条件下自动抑制，兼顾美观与可编程性